
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 2

Introdução à Programação em C

Exercícios (2.3 – 2.31)

Resolvidos passo a passo, com código C completo e saída esperada

Abordagem incremental: Capítulos 1 e 2

Contents

1	Exercício 2.3 – Instruções C Únicas	2
2	Exercício 2.4 – Instruções para Produto de Três Inteiros	3
3	Exercício 2.5 – Programa Completo: Produto de Três Inteiros	3
4	Exercício 2.6 – Identificar e Corrigir Erros	4
5	Exercício 2.7 – Identificar e Corrigir Erros (avançado)	6
6	Exercício 2.8 – Preenchimento de Lacunas	7
7	Exercício 2.9 – Instruções Únicas em C	8
8	Exercício 2.10 – Verdadeiro ou Falso	8
9	Exercício 2.11 – Mais Preenchimento de Lacunas	9
10	Exercício 2.12 – O que é exibido?	9
11	Exercício 2.13 – Variáveis com Valores Substituídos	9
12	Exercício 2.14 – A equação $y = ax^3 + 7$	10
13	Exercício 2.15 – Ordem de Avaliação e Valor de x	10
14	Exercício 2.16 – Aritmética Completa	11
15	Exercício 2.17 – Imprimindo 1 a 4	12
16	Exercício 2.18 – Comparando Inteiros	12
17	Exercício 2.19 – Aritmética, Maior e Menor	13
18	Exercício 2.20 – Círculo: Diâmetro, Circunferência e Área	14
19	Exercício 2.22 – O que imprime <code>printf("*\n**\n...");</code>	15
20	Exercício 2.24 – Par ou Ímpar	15
21	Exercício 2.26 – Múltiplos	16
22	Exercício 2.28 – Erro Fatal vs. Erro Não Fatal	17
23	Exercício 2.30 – Separando Dígitos de um Inteiro	17

1. Exercício 2.3 – Instruções C Únicas

Enunciado 2.3

Escreva uma única instrução C para cada alternativa: a) Declarar `c`, `estaVariavel`, `q76354` e `numero` como `int`. b) Pedir ao usuário que digite um inteiro, terminando com : . c) Ler um inteiro e armazenar em `a`. d) Se `numero != 7`, exibir "A variavel numero nao e igual a 7". e) Imprimir "Este e um programa em C." em uma linha. f) Imprimir em duas linhas, terminando a primeira em C. g) Imprimir cada palavra em uma linha separada. h) Imprimir as palavras separadas por tabulações.

Resposta detalhada

a) Declaração de múltiplas variáveis do tipo `int`

```
1 int c, estaVariavel, q76354, numero;
```

Múltiplas variáveis do mesmo tipo podem ser declaradas em uma única instrução, separadas por vírgulas. A instrução termina com ponto e vírgula.

b) Exibir prompt com dois pontos

```
1 printf( "Digite um inteiro: " );
```

Observe: não há `\n` ao final, pois o cursor deve ficar posicionado após o espaço, na mesma linha do prompt.

c) Ler um inteiro e armazenar em `a`

```
1 scanf( "%d", &a );
```

O `&` é o operador de endereço, obrigatório em `scanf` para informar ao compilador onde armazenar o valor lido.

d) Decisão com `if`

```
1 if ( numero != 7 ) {  
2     printf( "A variavel numero nao e igual a 7.\n"  
3         );  
3 }
```

O operador `!=` significa “diferente de”. O corpo do `if` executa apenas quando a condição for verdadeira.

e) Imprimir em uma linha

```
1 printf( "Este e um programa em C.\n" );
```

f) Imprimir em duas linhas (a primeira termina em C)

```
1 printf( "Este e um programa em\nC.\n" );
```

O `\n` no meio da string divide a saída em duas linhas. Saída:

Saída do programa

```
Este e um programa em  
C.
```

g) Cada palavra em uma linha separada

```
1 printf( "Este\ne\num\nprograma\nem\nC.\n" );
```

Cada `\n` separa uma palavra em sua própria linha.

h) Palavras separadas por tabulações

```
1 printf( "Este\te\tum\tprograma\tem\tC.\n" );
```

A sequência de escape `\t` insere uma tabulação horizontal.

2. Exercício 2.4 – Instruções para Produto de Três Inteiros

Enunciado 2.4

Escreva uma instrução (ou comentário) para: a) indicar o propósito do programa; b) declarar `x`, `y`, `z`, `resultado` como `int`; c) pedir três inteiros; d) ler três inteiros; e) calcular o produto e atribuir a `resultado`; f) exibir o resultado.

Resposta detalhada

a)

```
1 /* Calcula o produto de tres inteiros */
```

```
1 int x, y, z, resultado;
```

b)

```
1 printf( "Digite tres inteiros: " );
```

```
1 scanf( "%d%d%d", &x, &y, &z );
```

Um único `scanf` pode ler múltiplos valores – um especificador `%d` para cada variável, e o operador `&` antes de cada uma.

d)

```
1 resultado = x * y * z;
```

```
1 printf( "O produto e %d\n", resultado );
```

3. Exercício 2.5 – Programa Completo: Produto de Três Inteiros

Enunciado 2.5

Usando as instruções do Exercício 2.4, escreva um programa completo que calcule o produto de três inteiros.

Resposta detalhada

```
1  /* Figura Ex2.5: produto_tres_inteiros.c
2     Calcula o produto de tres inteiros */
3  #include <stdio.h>
4
5  int main( void )
6  {
7     int x, y, z, resultado; /* declara as quatro
8                             variaveis */
9
10    printf( "Digite tres inteiros: " ); /* prompt ao
11                                       usuario */
12    scanf( "%d%d%d", &x, &y, &z );      /* le tres
13                                       inteiros */
14
15    resultado = x * y * z;              /* calcula o
16                                       produto */
17
18    printf( "O produto e %d\n", resultado ); /* exhibe
19                                       resultado */
20
21    return 0; /* indica conclusao bem-sucedida */
22 } /* fim da funcao main */
```

Exemplo de execução:

Saída do programa

```
Digite tres inteiros: 4 5 6
O produto e 120
```

Passo a passo da execução:

- O programa inclui `<stdio.h>` para ter acesso a `printf` e `scanf`.
- Declara quatro variáveis `int`: `x`, `y`, `z`, `resultado`.
- Exibe o prompt e lê três inteiros do teclado.
- Calcula o produto: `resultado = 4 * 5 * 6 = 120`.
- Exibe o resultado formatado.
- Retorna 0 ao sistema operacional.

4. Exercício 2.6 – Identificar e Corrigir Erros

Enunciado 2.6

Identifique e corrija os erros em cada instrução:

- `printf( "O valor e %d\n", &numero );`
- `scanf( "%d%d", &numero1, numero2 );`
- `if ( c < 7 );{ printf( "C e menor que 7\n" ); }`
- `if ( c => 7 ) { printf( "C e igual ou menor que 7\n" ); }`

Resposta detalhada

- a) **Erro:** O & antes de `numero` é incorreto em `printf`. `printf` precisa do *valor* da variável, não de seu endereço.

Código com erro:

```
1 printf( "0 valor e %d\n", &numero ); /* ERRO: &
    desnecessario */
```

Código corrigido:

```
1 printf( "0 valor e %d\n", numero ); /* CORRETO:
    sem & */
```

- b) **Erro:** A variável `numero2` está sem o operador de endereço & em `scanf`.

Código com erro:

```
1 scanf( "%d%d", &numero1, numero2 ); /* ERRO:
    falta & em numero2 */
```

Código corrigido:

```
1 scanf( "%d%d", &numero1, &numero2 ); /* CORRETO */
```

- c) **Erro:** Ponto e vírgula após o parêntese direito da condição do `if`. Esse ; cria uma *instrução vazia* como corpo do `if`, de modo que o `printf` sempre é executado, independente da condição.

Código com erro:

```
1 if ( c < 7 );{ /* ERRO: ;
    cria corpo vazio */
2     printf( "C e menor que 7\n" );
3 }
```

Código corrigido:

```
1 if ( c < 7 ) { /* CORRETO
    : sem ; apos ) */
2     printf( "C e menor que 7\n" );
3 }
```

- d) **Erro:** O operador relacional `=>` não existe em C. O operador correto para “maior ou igual a” é `>=`.

Código com erro:

```
1 if ( c => 7 ) { /* ERRO:
    => nao existe em C */
2     printf( "C e igual ou maior que 7\n" );
3 }
```

Código corrigido:

```
1 if ( c >= 7 ) { /* CORRETO
    : >= (maior ou igual) */
2     printf( "C e igual ou maior que 7\n" );
3 }
```

5. Exercício 2.7 – Identificar e Corrigir Erros (avanzado)

Enunciado 2.7

Identifique e corrija os erros (pode haver mais de um por instrução).

Resposta detalhada

a) **Código:** `scanf( "d", valor );`

Erros: (1) Falta o % no especificador de conversão; (2) falta & antes de valor.

```
1 scanf( "%d", &valor ); /* CORRETO */
```

b) **Código:** `printf( "O produto de %d e %d e %d\n", x, y );`

Erros: (1) O \n está *fora* das aspas da string; (2) falta o terceiro argumento para o terceiro %d.

```
1 printf( "O produto de %d e %d e %d\n", x, y, x * y
); /* CORRETO */
```

c) **Código:** `primeiroNumero + segundoNumero = somaDosNumeros`

Erros: (1) O cálculo está no lado esquerdo do = (atribuição inválida); (2) falta ponto e vírgula.

```
1 somaDosNumeros = primeiroNumero + segundoNumero; /*
CORRETO */
```

d) **Código:** `if ( numero => maior ) maior == numero;`

Erros: (1) => não existe em C – deve ser >; (2) == é comparação, não atribuição – deve ser =.

```
1 if ( numero >= maior ) {
2     maior = numero; /* CORRETO */
3 }
```

e) **Código:** `/* Programa para determinar o maior dentre tres inteiros */`

Erro: Comentário com delimitadores invertidos. O comentário deve começar com /* e terminar com */.

```
1 /* Programa para determinar o maior dentre tres
inteiros */ /* CORRETO */
```

f) **Código:** `Scanf( "%d", umInteiro );`

Erros: (1) Scanf com S maiúsculo é inválido (C é *case-sensitive*); (2) falta & antes de umInteiro.

```
1 scanf( "%d", &umInteiro ); /* CORRETO */
```

g) **Código:** `printf( "Modulo de %d dividido por %d e\n", x, y, x % y );`

Erro: O \n está no lugar errado. A frase fica incompleta antes da nova linha.

```
1 printf( "Modulo de %d dividido por %d e %d\n", x, y
    , x % y ); /* CORRETO */
```

- h) **Código:** `if ( x = y ); printf( %d e igual a %d\n", x, y );`
Erros: (1) `x = y` é atribuição, não comparação – deve ser `x == y`; (2) ponto e vírgula após `)` cria corpo vazio; (3) falta aspas de abertura na string de `printf`.

```
1 if ( x == y ) {
2     printf( "%d e igual a %d\n", x, y ); /* CORRETO
3     */
4 }
```

- i) **Código:** `print( "A soma e %d\n," x + y );`
Erros: (1) `print` não existe – deve ser `printf`; (2) a vírgula está dentro das aspas (deve estar fora).

```
1 printf( "A soma e %d\n", x + y ); /* CORRETO */
```

- j) **Código:** `printf( "A soma e %d\n," x + y );`
Erro: A vírgula está dentro das aspas – deve estar fora, separando os argumentos.

```
1 printf( "A soma e %d\n", x + y ); /* CORRETO */
```

- k) **Código:** `Printf( "O valor que voce digitou e: %d\n, &valor );`
Erros: (1) `Printf` com P maiúsculo é inválido; (2) as aspas de fechamento da string estão faltando antes de `, &valor`; (3) `&valor` não deve ser usado em `printf`.

```
1 printf( "O valor que voce digitou e: %d\n", valor )
    ; /* CORRETO */
```

6. Exercício 2.8 – Preenchimento de Lacunas

Exercício 2.8 – Respostas

- a) **Comentários** são usados para documentar um programa e melhorar sua legibilidade.
- b) A função usada para exibir informações na tela é `printf`.
- c) Uma instrução C responsável pela tomada de decisão é `if`.
- d) Os cálculos normalmente são realizados por instruções de **atribuição** (usando o operador `=`).
- e) A função `scanf` lê valores do teclado.

7. Exercício 2.9 – Instruções Únicas em C

Exercício 2.9

a) Exibir mensagem:

```
1 printf( "Digite dois numeros.\n" );
```

b) Atribuir produto de b e c a a:

```
1 a = b * c;
```

c) Indicar cálculo de folha de pagamento (comentário):

```
1 /* Calcula o salario liquido a partir do salario  
   bruto e descontos */
```

d) Ler três inteiros para a, b e c:

```
1 scanf( "%d%d%d", &a, &b, &c );
```

8. Exercício 2.10 – Verdadeiro ou Falso

Exercício 2.10

a) “Os operadores em C são avaliados da esquerda para a direita.”

Falso. Os operadores são avaliados de acordo com sua *precedência* e *associatividade*. A maioria se associa da esquerda para a direita, mas o operador de atribuição = se associa da *direita para a esquerda*. Além disso, *, /, % têm precedência sobre +, -, independente da posição.

b) Nomes de variáveis válidos: _sub_barra_, m928134, t5, j7, suas_vendas, total_sua_conta, a, b, c, z, z2.

Verdadeiro. Todos são identificadores válidos em C. Um identificador pode conter letras, dígitos e sublinhados, mas não pode começar com um dígito. Todos esses exemplos atendem a essa regra.

c) printf("a = 5;"); é uma instrução de atribuição.

Falso. Essa é uma *instrução de saída* (printf). Ela apenas *exibe* o texto a = 5; na tela. Uma instrução de atribuição real seria a = 5; – sem as aspas e sem printf.

d) Uma expressão sem parênteses é avaliada da esquerda para a direita.

Falso. A avaliação segue as regras de precedência primeiro. Por exemplo, 2 + 3 * 4 é avaliado como 2 + 12 = 14 (e não como 5 * 4 = 20), porque * tem precedência maior. A associatividade da esquerda para a direita só se aplica entre operadores de *mesmo* nível de precedência.

e) Nomes de variáveis inválidos: 3g, 87, 67h2, h22, 2h.

Verdadeiro. Todos começam com um dígito, o que é proibido em C. Um identificador válido deve começar com uma letra ou sublinhado. Note que h22 e h22h seriam válidos, pois começam com letra.

9. Exercício 2.11 – Mais Preenchimento de Lacunas

Exercício 2.11

- a) As operações no mesmo nível de precedência da multiplicação são: **divisão (/) e módulo (%)**. Juntos, *, / e % formam o grupo de maior precedência aritmética.
- b) Quando os parênteses são aninhados, o par **mais interno** é avaliado primeiro. Exemplo: em ((a + b) + c), a + b é calculado primeiro.
- c) Uma posição na memória que pode conter diferentes valores ao longo da execução é chamada de **variável**.

10. Exercício 2.12 – O que é exibido?

Enunciado 2.12

O que é exibido por cada instrução? Considere $x = 2$ e $y = 3$.

Resposta detalhada

- a) `printf( "%d", x );` → exibe **2**
- b) `printf( "%d", x + x );` → $2 + 2 = 4$ → exibe **4**
- c) `printf( "x=" );` → exibe a string literal **x=**
- d) `printf( "x=%d", x );` → exibe **x=2**
- e) `printf( "%d = %d", x + y, y + x );` → $5 = 5$ → exibe **5 = 5**
- f) `z = x + y;` → **Nada** é exibido; apenas atribui 5 a z
- g) `scanf( "%d%d", &x, &y );` → **Nada** é exibido; aguarda entrada do usuário
- h) `/* printf( "x + y = %d", x + y ); */` → **Nada**; está comentado
- i) `printf( "\n" );` → **Nada visível**; apenas move o cursor para nova linha

11. Exercício 2.13 – Variáveis com Valores Substituídos

Exercício 2.13

- a) `scanf( "%d%d%d%d%d", &b, &c, &d, &e, &f );` → **Sim**: b, c, d, e, f recebem novos valores (escrita destrutiva)
- b) `p = i + j + k + 7;` → **Sim**: p recebe um novo valor
- c) `printf( "Valores sao substituidos" );` → **Não**: apenas exibe uma string; nenhuma variável é alterada
- d) `printf( "a = 5" );` → **Não**: apenas exibe o texto; a não é alterada

12. Exercício 2.14 – A equação $y = ax^3 + 7$

Exercício 2.14 – Instruções corretas para $y = ax^3 + 7$

A equação algébrica $y = ax^3 + 7$ deve ser expressa em C sem usar a notação de expoente. Como não há operador de exponenciação em C (cap. 2), representamos x^3 como $x*x*x$.

a) $y = a * x * x * x + 7$; → **CORRETO**

Avaliação: $a \cdot x \cdot x \cdot x$ (multiplicações da esquerda) $+ 7$

b) $y = a * x * x * (x + 7)$; → **INCORRETO**

Calcula $ax^2(x + 7) = ax^3 + 7ax^2 \neq ax^3 + 7$

c) $y = (a * x) * x * (x + 7)$; → **INCORRETO**

Mesma razão do item b.

d) $y = (a * x) * x * x + 7$; → **CORRETO**

$(ax) \cdot x \cdot x + 7 = ax^3 + 7$

e) $y = a * (x * x * x) + 7$; → **CORRETO**

$a \cdot (x^3) + 7 = ax^3 + 7$

f) $y = a * x * (x * x + 7)$; → **INCORRETO**

$ax(x^2 + 7) = ax^3 + 7ax \neq ax^3 + 7$

Respostas corretas: a), d) e e).

13. Exercício 2.15 – Ordem de Avaliação e Valor de x

Exercício 2.15

a) $x = 7 + 3 * 6 / 2 - 1$;

Passo a passo (da esquerda para a direita, * e / antes de + e -):

$$7 + \underbrace{3 \times 6}_{18} / 2 - 1 \rightarrow 7 + \underbrace{18 / 2}_9 - 1 \rightarrow \underbrace{7 + 9}_{16} - 1 \rightarrow 16 - 1 = 15$$

b) $x = 2 \% 2 + 2 * 2 - 2 / 2$;

$$\underbrace{2 \% 2}_0 + \underbrace{2 \times 2}_4 - \underbrace{2 / 2}_1 \rightarrow 0 + 4 - 1 = 3$$

c) $x = (3 * 9 * (3 + (9 * 3 / (3))))$;

Do parêntese mais interno para o externo:

$$9 * 3 / 3 = 27 / 3 = 9 \rightarrow 3 + 9 = 12 \rightarrow 3 * 9 * 12 = 27 * 12 = 324$$

14. Exercício 2.16 – Aritmética Completa

Exercício 2.16 – Programa de aritmética

```
1  /* Ex2.16: aritmetica.c
2     Le dois numeros e imprime soma, produto, diferenca,
3     quociente e modulo */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9      int a, b; /* dois inteiros fornecidos pelo usuario
10              */
11
12     printf( "Digite dois inteiros: " );
13     scanf( "%d%d", &a, &b );
14
15     printf( "Soma:      %d + %d = %d\n",  a, b, a + b
16           );
17     printf( "Produto:   %d * %d = %d\n",  a, b, a * b
18           );
19     printf( "Diferenca: %d - %d = %d\n",  a, b, a - b
20           );
21
22     /* Divisao inteira: o resultado descarta a parte
23        fracionaria */
24     printf( "Quociente: %d / %d = %d\n",  a, b, a / b
25           );
26
27     /* Modulo: resto da divisao inteira */
28     printf( "Modulo:    %d %% %d = %d\n", a, b, a % b
29           );
30
31     return 0;
32 } /* fim da funcao main */
```

Saída do programa

```
Digite dois inteiros: 17 5
Soma: 17 + 5 = 22
Produto: 17 * 5 = 85
Diferenca: 17 - 5 = 12
Quociente: 17 / 5 = 3
Modulo: 17 % 5 = 2
```

Nota: Para exibir o caractere % literalmente com printf, usa-se %%.

15. Exercício 2.17 – Imprimindo 1 a 4

Exercício 2.17 – Três métodos para imprimir 1 2 3 4

a) Uma instrução printf sem especificadores:

```
1 printf( "1 2 3 4\n" );
```

b) Uma instrução com quatro especificadores:

```
1 printf( "%d %d %d %d\n", 1, 2, 3, 4 );
```

c) Quatro instruções printf:

```
1 printf( "%d ", 1 );  
2 printf( "%d ", 2 );  
3 printf( "%d ", 3 );  
4 printf( "%d\n", 4 );
```

Saída do programa

```
1 2 3 4
```

16. Exercício 2.18 – Comparando Inteiros

Exercício 2.18 – Comparar dois inteiros

```
1  /* Ex2.18: comparando_inteiros.c  
2     Le dois inteiros e imprime o maior, ou "iguais" */  
3  #include <stdio.h>  
4  
5  int main( void )  
6  {  
7      int num1, num2; /* dois inteiros a comparar */  
8  
9      printf( "Digite dois inteiros: " );  
10     scanf( "%d%d", &num1, &num2 );  
11  
12     if ( num1 > num2 ) {  
13         printf( "%d e maior\n", num1 );  
14     }  
15  
16     if ( num2 > num1 ) {  
17         printf( "%d e maior\n", num2 );  
18     }  
19  
20     if ( num1 == num2 ) {  
21         printf( "Esses numeros sao iguais\n" );  
22     }  
23
```

```
24     return 0;
25 } /* fim da funcao main */
```

Saída do programa

```
Digite dois inteiros: 8 3
8 e maior
```

17. Exercício 2.19 – Aritmética, Maior e Menor

Exercício 2.19 – Soma, média, produto, menor e maior

```
1  /* Ex2.19: maior_menor.c
2     Le tres inteiros e calcula soma, media, produto,
3     menor e maior */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9      int n1, n2, n3;           /* tres inteiros */
10     int soma, produto;
11     int menor, maior;
12
13     printf( "Digite tres inteiros diferentes: " );
14     scanf( "%d%d%d", &n1, &n2, &n3 );
15
16     soma      = n1 + n2 + n3;
17     produto   = n1 * n2 * n3;
18     menor     = n1; /* assume n1 como menor inicial */
19     maior     = n1; /* assume n1 como maior inicial */
20
21     /* Verifica se n2 e menor ou maior */
22     if ( n2 < menor ) menor = n2;
23     if ( n2 > maior ) maior = n2;
24
25     /* Verifica se n3 e menor ou maior */
26     if ( n3 < menor ) menor = n3;
27     if ( n3 > maior ) maior = n3;
28
29     printf( "A soma e %d\n", soma );
30     printf( "A media e %d\n", soma / 3 ); /*
31         divisao inteira */
32     printf( "O produto e %d\n", produto );
33     printf( "O menor e %d\n", menor );
34     printf( "O maior e %d\n", maior );
35
36     return 0;
37 } /* fim da funcao main */
```

Saída do programa

```
Digite tres inteiros diferentes: 13 27 14
A soma e 54
A media e 18
O produto e 4914
O menor e 13
O maior e 27
```

Nota sobre a média: Como usamos apenas `int` (Cap. 2), a divisão $54 / 3 = 18$ é exata. Com valores que não dividem exatamente, a parte decimal seria truncada. No Capítulo 3 aprenderemos `double` para precisão.

18. Exercício 2.20 – Círculo: Diâmetro, Circunferência e Área

Exercício 2.20 – Propriedades do círculo

```
1  /* Ex2.20: circulo.c
2     Calcula diametro, circunferencia e area de um
3     circulo */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9     int raio; /* raio em unidades inteiras */
10
11     printf( "Digite o raio do circulo: " );
12     scanf( "%d", &raio );
13
14     /* Calcula dentro dos printf, usando %f para ponto
15     flutuante */
16     printf( "Diametro:      %f\n", 2 * 3.14159 * raio
17           / 3.14159 );
18     printf( "Diametro:      %f\n", (float)( 2 * raio )
19           );
20     printf( "Circunferencia: %f\n", 2 * 3.14159 * raio
21           );
22     printf( "Area:          %f\n", 3.14159 * raio *
23           raio );
24
25     return 0;
26 } /* fim da funcao main */
```

Nota: O enunciado pede que cada cálculo seja feito dentro de `printf` e use `%f`. No Cap. 2, trabalhamos com `int`; ao multiplicar um `int` por uma constante `double` (como 3.14159), o resultado automaticamente torna-se `double` – por isso `%f` funciona aqui.

Saída do programa

```

Digite o raio do circulo: 5
Diametro: 10.000000
Circunferencia: 31.415900
Area: 78.539750

```

19. Exercício 2.22 – O que imprime `printf("*\n**\n...\n");`

Exercício 2.22

O código `printf( "\n**\n***\n****\n*****\n" );` imprime:

Saída do programa

```

*
**
***
****
*****

```

Cada `\n` provoca um salto de linha. A quantidade de asteriscos aumenta de 1 a 5.

20. Exercício 2.24 – Par ou Ímpar

Exercício 2.24 – Par ou ímpar com operador módulo

```

1  /* Ex2.24: par_impar.c
2     Determina se um inteiro é par ou ímpar */
3  #include <stdio.h>
4
5  int main( void )
6  {
7      int numero;
8
9      printf( "Digite um inteiro: " );
10     scanf( "%d", &numero );
11
12     /* Um número é par se o resto da divisão por 2 é
13        zero */
14     if ( numero % 2 == 0 ) {
15         printf( "%d é PAR\n", numero );
16     }
17
18     if ( numero % 2 != 0 ) {
19         printf( "%d é IMPAR\n", numero );
20     }

```



```
20
21     return 0;
22 } /* fim da funcao main */
```

Saída do programa

```
Digite um inteiro: 14
14 e PAR
```

21. Exercício 2.26 – Múltiplos

Exercício 2.26 – Verificar se o primeiro é múltiplo do segundo

```
1  /* Ex2.26: multiplos.c
2     Verifica se o primeiro inteiro e multiplo do
3     segundo */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9      int a, b;
10
11      printf( "Digite dois inteiros: " );
12      scanf( "%d%d", &a, &b );
13
14      /* a e multiplo de b se o resto de a/b for zero */
15      if ( a % b == 0 ) {
16          printf( "%d e multiplo de %d\n", a, b );
17      }
18
19      if ( a % b != 0 ) {
20          printf( "%d NAO e multiplo de %d\n", a, b );
21      }
22
23      return 0;
24 } /* fim da funcao main */
```

Saída do programa

```
Digite dois inteiros: 15 5
15 e multiplo de 5
```

22. Exercício 2.28 – Erro Fatal vs. Erro Não Fatal

Exercício 2.28 – Distinguindo erros fatais de não fatais

Da Seção 1.14 e Seção 2.5:

Erro fatal: Um erro que causa o *encerramento imediato* do programa, sem que ele complete sua tarefa. Exemplo clássico: divisão por zero ($x / 0$). O programa é interrompido abruptamente pelo sistema operacional.

Erro não fatal: Um erro que permite que o programa *continue executando* até o fim, mas produzindo *resultados incorretos*. Exemplo: esquecer o `&` em um `scanf` pode gerar resultados errados sem terminar o programa com um crash.

Por que preferir erros fatais?

Paradoxalmente, um erro fatal é mais fácil de encontrar e corrigir: o programa para imediatamente e frequentemente indica a linha onde ocorreu o problema. Um erro não fatal pode passar despercebido por muito tempo, pois o programa aparentemente “funciona”, mas produz respostas erradas que só serão descobertas com testes cuidadosos – possivelmente muito depois do produto estar em uso real.

23. Exercício 2.30 – Separando Dígitos de um Inteiro

Exercício 2.30 – Separar dígitos de número de 5 dígitos

```
1  /* Ex2.30: separar_digitos.c
2     Le um numero de 5 digitos e os separa por tres
3     espacos */
4
5  #include <stdio.h>
6
7  int main( void )
8  {
9      int numero, d1, d2, d3, d4, d5;
10
11      printf( "Digite um numero de 5 digitos: " );
12      scanf( "%d", &numero );
13
14      /* Extraindo cada digito usando divisao inteira e
15      modulo */
16      d1 = numero / 10000;          /* milhar: 42139 /
17      10000 = 4 */
18      d2 = numero / 1000 % 10;      /* centena: 42139 /
19      1000 = 42 % 10 = 2 */
20      d3 = numero / 100 % 10;       /* dezena: 421 % 10 =
21      1 */
22      d4 = numero / 10 % 10;        /* unidade de dezena:
23      4213 % 10 = 3 */
24      d5 = numero % 10;             /* unidade: 42139 %
25      10 = 9 */
```

```
18  
19     printf( "%d    %d    %d    %d    %d\n", d1, d2, d3, d4  
20             , d5 );  
21     return 0;  
22 } /* fim da funcao main */
```

Saída do programa

```
Digite um numero de 5 digitos: 42139  
4 2 1 3 9
```